

LAPORAN TUGAS BESAR STRATEGI ALGORITMA

IMPLEMENTASI ALGORITMA GREEDY PADA BOT ENAA DALAM PERMAINAN ROBOCODE TANK ROYALE



Oleh: Kelompok ENAA

Adelia Eva Ananta (124140123)

Eleora Arga Curie Gurning (124140222)

Luthfi Daffa Nabila (122140042)

Dosen Pengampu: Imam Ekowicaksono, S.Si., M.Si.

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA**

2026

DAFTAR ISI

DAFTAR ISI	i
BAB I DESKRIPSI TUGAS.....	1
1.1 Komponen Permainan	1
1.2 Sistem Skor	2
1.3 Spesifikasi Tugas.....	3
BAB II LANDASAN TEORI	4
2.1 Dasar Teori Algoritma Greedy	4
2.2 Cara Kerja Program	5
2.2.1 Arsitektur Bot Robocode Tank Royale	5
2.2.2 Implementasi Algoritma Greedy ke dalam Bot	5
2.2.3 Cara Menjalankan Bot.....	8
2.2.4 Kesimpulan.....	9
BAB III APLIKASI STRATEGI GREEDY	10
3.1 Proses Mapping Persoalan ke Elemen Greedy	10
3.2 Eksplorasi Empat Alternatif Solusi Greedy.....	11
3.2.1 Bot Utama	11
3.2.2 Bot Alternatif 1	12
3.2.3 Bot Alternatif 2.....	13
3.2.4 Bot Alternatif 3.....	15
3.3 Analisis Efisiensi dan Efektivitas	16
3.4 Strategi Greedy yang Dipilih.....	16
BAB IV IMPLEMENTASI DAN PENGUJIAN.....	19
4.1 Implementasi Algoritma Greedy.....	19
4.1.1 Prinsip Desain ENAA_Nemesis	19
4.1.2 Pseudocode Detail ENAA_Nemesis.....	20
4.1.3 Penjelasan Komponen Kunci Implementasi ENAA_Nemesis	23
4.2 Struktur Data yang Digunakan dalam Program Bot ENAA_Nemesis	26
4.3 Analisis Solusi Algoritma Greedy Melalui Skenario Pengujian.....	27
4.3.1 Skenario Pengujian.....	27
4.3.2 Resume Keunggulan dan Kelemahan Bot ENAA_Nemesis.....	35

BAB V KESIMPULAN DAN SARAN	39
5.1 Kesimpulan	39
5.2 Saran.....	40
LAMPIRAN.....	43
A. Repository Github	43
B. Video Penjelasan	43
DARFAR PUSTAKA	44

BAB I

DESKRIPSI TUGAS

Robocode Tank Royale merupakan permainan berbasis kompetisi di mana setiap pemain akan menulis kode untuk bot berupa tank virtual yang bertarung melawan bot lain di arena. Istilah "Robocode" adalah akronim dari "Robot code", yang diambil dari versi awal permainan ini. Robocode Tank Royale adalah pengembangan dari versi sebelumnya, di mana bot dapat ikut serta melalui internet.

Pertempuran terjadi dengan format Battle Royale, di mana bot-bot berada hingga hanya satu bot yang tersisa sebagai pemenang. Pemain bertindak sebagai pengembang bot dan tidak memiliki pengaruh langsung terhadap alur pertempuran. Program yang dibuat memuat petunjuk mengenai cara bot bergerak, mengidentifikasi musuh, menembakkan senjata, dan merespons berbagai kejadian.

1.1 Komponen Permainan

- Rounds dan Turns: Pertarungan terdiri dari beberapa rounds, yang masing-masing terbagi menjadi turns sebagai unit waktu terkecil. Pada setiap giliran, bot bisa bergerak, memindai, dan menembak.
- Batas Waktu Giliran: Setiap bot memiliki batas waktu giliran antara 30–50 ms. Apabila bot tidak mengeluarkan perintah tepat waktu, maka giliran tersebut akan dilewatkan.
- Energi: Setiap bot memiliki awal 100 poin energi. Bot kehilangan tenaga jika terkena tembakan atau bertabrakan. Bot memperoleh energi sebesar $3 \times$ daya tembak saat pelurunya mengenai target.
- Proyektil: Semakin tinggi daya ledak, proyektil menjadi semakin berat, lambat, dan destruktif.
- Suhu Meriam (Pan Panas): Setiap tembakan menghasilkan suhu. Bot tidak dapat menembak sampai suhu meriam mencapai nol.
- Tabrakan: Bot akan mengalami kerusakan jika menghantam dinding atau bot lainnya. Menabrak lawan saat bergerak ke depan disebut ramming.
- Komponen Tank: Terdiri dari Badan, Senjata, dan Radar yang dapat berputar secara terpisah.

- Gerakan: Akselerasi maksimum 1 unit per putaran, deselerasi maksimum 2 unit/putaran.
- Berbelok: Radar maksimum 45° per putaran, senjata/menara maksimum 20°/putaran, badan maksimum 10° per putaran (tergantung kecepatan).
- Pemindaian: Radar mampu mendeteksi musuh sejauh 1200 piksel dalam sudut sapuan (scan arc).

1.2 Sistem Skor

Skor akhir bot merupakan akumulasi dari enam komponen berikut:

No	Komponen Skor	Deskripsi	Nilai
1	Bullet Damage	Poin yang dihasilkan sebesar damage yang diberikan ke musuh melalui peluru	= damage
2	Bullet Damage Bonus	Bonus yang dihasilkan saat peluru membunuh musuh	= 20% × total damage
3	Survival Score	Setiap bot lain mati, semua bot yang masih hidup mendapat poin	50 poin
4	Last Survival Bonus	Bot terakhir yang dapat bertahan dalam satu ronde	10 × jumlah musuh
5	Ram Damage	Poin yang dihasilkan saat menabrak musuh (ramming)	= 2 × damage tabrakan

6	Ram Damage Bonus	Bonus saat musuh terbunuh karena ditabrak	$= 30\% \times \text{total damage}$
---	------------------	---	-------------------------------------

1.3 Spesifikasi Tugas

1. Mengembangkan empat robot (satu utama dan tiga alternatif) menggunakan bahasa C# (.NET) yang menerapkan algoritma Greedy dalam kompetisi Robocode Tank Royale.
2. Setiap bot akan mengadopsi strategi greedy yang berbeda, masing-masing dengan pendekatan heuristik yang unik.
3. Strategi greedy perlu berhubungan dengan fungsi tujuan permainan, yaitu mencapai angka skor maksimum.
4. Bot tidak boleh identik dengan contoh yang ada dalam paket awal maupun dalam repositori engine asli
5. Setiap pendekatan greedy harus diuraikan dalam laporan dan dibuktikan selama presentasi.

Game Engine yang digunakan adalah versi yang telah disesuaikan oleh asisten, dengan modifikasi: antarmuka tema terang, tampilan skor dan energi di samping arena, serta Batas Giliran yang bisa diatur di menu "atur aturan".

BAB II

LANDASAN TEORI

2.1 Dasar Teori Algoritma Greedy

Algoritma Greedy merupakan paradigma dalam perancangan algoritma yang menyelesaikan permasalahan optimasi dengan cara memilih opsi yang tampak paling menguntungkan (optimum lokal) di setiap tahap pengambilan keputusan, dengan harapan akumulasi pilihan tersebut akan menghasilkan solusi yang optimal secara keseluruhan (optimum global).

Istilah "Greedy" menggambarkan cara kerjanya: algoritma ini selalu memilih keuntungan maksimum yang ada saat ini tanpa memikirkan konsekuensi di masa depan. Setelah keputusan dibuat, algoritma greedy tidak akan melakukan penelusuran kembali atau mengubah keputusan itu.

Menurut Cormen et al. (2009) di dalam Introduction to Algorithms, algoritma greedy terdiri dari lima elemen utama:

- Kumpulan Kandidat (C): sekumpulan elemen yang bisa dipilih sebagai solusi.
- Fungsi Seleksi: menentukan elemen terbaik dari sekumpulan kandidat di setiap langkah.
- Fungsi Kelayakan: mengevaluasi apakah elemen yang dipilih cocok untuk dimasukkan ke dalam solusi.
- Fungsi Solusi: mengevaluasi apakah sekumpulan elemen yang dipilih telah membentuk solusi yang utuh.
- Fungsi Objektif: nilai yang ingin ditingkatkan atau diturunkan.

Algoritma greedy tidak menjamin solusi global optimal untuk setiap jenis masalah. Namun, pada masalah tertentu seperti Minimum Spanning Tree (Kruskal, Prim), Shortest Path (Dijkstra), dan penjadwalan aktivitas algoritma greedy terbukti memberikan solusi yang optimal. Untuk permasalahan lain, metode greedy berperan sebagai heuristik yang menghasilkan solusi yang memuaskan dengan tingkat kompleksitas waktu yang rendah.

2.2 Cara Kerja Program

2.2.1 Arsitektur Bot Robocode Tank Royale

Bot dikembangkan dengan bahasa C# menggunakan NET SDK dan API Robocode Tank Royale Bot. Setiap bot adalah class yang mewarisi class Bot dari API, dan mengimplementasikan dua komponen utama:

- Method Run(): siklus utama yang beroperasi tanpa henti selama bot aktif. Di sini logika pergerakan standar ditetapkan, mencakup rotasi tubuh, radar, dan pengaturan kecepatan.
- Event Handlers: metode yang dipanggil secara otomatis oleh mesin game ketika peristiwa tertentu terjadi, seperti OnScannedBot(), OnHitByBullet(), OnHitBot(), OnHitWall(), dan OnBotDeath()

Pada setiap turn, bot melakukan:

1. Mengakses data terkini dari mesin permainan (letak, energi, orientasi, dan lain-lain).
2. Menganalisis situasi saat ini berdasarkan strategi greedy yang diterapkan.
3. Menentukan tindakan terbaik secara lokal (greedy decision).
4. Mengirimkan perintah aksi ke game engine melalui method Go().

2.2.2 Implementasi Algoritma Greedy ke dalam Bot

Pada ENAA_Nemesis, algoritma greedy diterapkan melalui empat mekanisme keputusan utama yang dijalankan setiap giliran. Di bawah ini adalah penjabaran rinci mengenai setiap mekanisme tersebut.

a. First-Lock Target Selection (Pemilihan Target Pertama)

Bot menyimpan ID musuh yang pertama kali terdeteksi oleh radar ke dalam variabel `_lockedBotId`. Setelah variabel ini terisi, bot tidak akan mengubah target meskipun musuh lain masuk dalam jangkauan radar. Setiap lawan yang terdeteksi dibandingkan dengan `_lockedBotId`, dan jika tidak cocok maka data pemindaian itu diabaikan sepenuhnya.

Ini adalah keputusan greedy murni: bot memilih target pertama yang ada tanpa menilai semua musuh di arena. Pendekatan ini mengonsumsi Bullet Damage Bonus (20% dari total damage pada satu target) dengan cara yang lebih efisien dibandingkan dengan berpindah-pindah target. Target dianggap hilang jika tidak ada scan yang cocok selama lebih dari

LOST_TARGET_TIMEOUT (10 turn), setelah itu bot melakukan pemindaian ulang 360° secara penuh untuk menemukan target baru.

b. Adaptive Firepower Selection (Pemilihan Firepower Adaptif)

Bot memilih nilai firepower secara greedy dengan mempertimbangkan jarak ke target saat melakukan penembakan. Kriteria pemilihannya adalah sebagai berikut:

1. Jarak < 150 piksel = Firepower 3.0 (maksimum, peluru berat dan menghancurkan)
2. Jarak 150 - 400 piksel = Firepower 2.5 (penyeimbangan antara damage dan kecepatan peluru)
3. Jarak > 400 piksel = Firepower 2.0 (efisiensi energi, peluru lebih cepat)

Strategi ini meningkatkan rasio damage per energi, ketika musuh dekat, daya tembak yang dihasilkan tinggi dan menjamin damage besar per peluru. Ketika musuh jauh, daya tembak yang dihasilkan lebih rendah dengan tujuan untuk menjaga kecepatan peluru agar lebih mudah mengenai target yang bergerak, sekaligus mengurangi konsumsi energi bot.

c. Predictive Firing (Tembak Prediktif)

Dibandingkan menembak langsung ke posisi musuh saat ini, bot memperkirakan posisi musuh beberapa langkah ke depan. Perhitungannya memakai kecepatan dan arah pergerakan musuh yang terakhir terdata dari OnScannedBot, digabungkan dengan waktu perjalanan peluru (tergantung pada jarak dan daya tembak yang dipilih).

Gun kemudian diarahkan menuju titik perkiraan itu, dan tembakan dilakukan hanya jika sudut antara arah gun dan arah ke titik perkiraan berada dalam batas toleransi $\leq 15^\circ$. Metode ini secara signifikan meningkatkan tingkat keberhasilan dibandingkan menembak ke lokasi saat ini, terutama pada jarak lebih dari 200 piksel di mana lawan memiliki cukup waktu untuk bergerak sebelum peluru sampai. Ini merupakan keputusan greedy: bot tidak

memikirkan kemungkinan lawan berbalik arah, tetapi segera membuat pilihan berdasarkan informasi terbaik yang ada pada saat itu.

d. Radar Infinity Lock (Kunci Radar Permanen)

Setiap kali OnScannedBot aktif pada target yang sudah terkunci, bot akan menghitung radarBearing, yaitu selisih antara arah radar saat ini dengan posisi target. Setelah itu, radar diputar sebesar $1.5 \times \text{radarBearing}$ ke arah target. Pengali 1.5 digunakan agar radar bergerak sedikit melebihi posisi target, sehingga target tetap berada dalam area pemindaian meskipun sedang bergerak. Dengan cara ini, radar dapat terus mengikuti pergerakan target secara lebih stabil dan konsisten.

Tanpa metode ini, radar dapat kehilangan target dalam beberapa giliran karena sudut pemindaian radar terbatas, yaitu maksimal 45° per giliran. Dengan teknik Infinity Lock, radar terus menjaga target tetap terpantau sehingga bot memperoleh informasi posisi dan kecepatan musuh di setiap giliran tanpa terputus. Informasi yang konsisten ini membuat estimasi posisi musuh pada mekanisme Predictive Firing menjadi lebih akurat. Strategi ini termasuk pendekatan serakah (greedy), karena bot memprioritaskan mempertahankan informasi target yang sedang terdeteksi dibanding memindai seluruh area arena.

e. Circular Movement (Gerakan Melingkar Terus-Menerus)

Dalam setiap iterasi pada loop Run(), bot menjalankan SetTurnLeft(1.7) bersamaan dengan SetForward(10000) serta mengatur MaxSpeed = 6. Kombinasi ini menghasilkan pola pergerakan melingkar yang stabil dan terus-menerus. Nilai SetForward yang sangat besar digunakan agar bot tetap bergerak maju tanpa perlu sering mengganti perintah gerak, sehingga pergerakannya menjadi lebih konsisten selama pertandingan berlangsung.

Gerakan berputar ini memberi dua keuntungan sekaligus: bot dapat mendekati lawan secara spiral sehingga peluang menghasilkan damage lebih besar pada jarak dekat meningkat, sekaligus membuat bot lebih sulit dibidik karena arah geraknya terus berubah. Bot juga tidak melakukan perencanaan jalur jangka panjang maupun penghindaran rintangan secara

kompleks. Pada setiap iterasi, keputusan yang diambil selalu sama, yaitu terus bergerak maju sambil berbelok ke kiri, tanpa mempertimbangkan kondisi arena secara mendalam.

2.2.3 Cara Menjalankan Bot

Untuk menjalankan bot, beberapa langkah perlu dilakukan mulai dari persiapan environment hingga bot aktif bertarung di arena. Berikut adalah tahapan menjalankan ENAA_Nemesis:

1. **Persiapan Environment:** Pastikan .NET SDK sudah terpasang pada sistem operasi yang digunakan (Windows/Linux/macOS). Setelah itu, unduh dan ekstrak paket awal game engine yang telah dimodifikasi oleh asisten. Paket tersebut sudah mencakup GUI bertema terang, tampilan skor dan energi di samping arena, serta fitur Batas Giliran pada menu “Atur Aturan”. Download dan ekstrak starter pack game engine yang telah dimodifikasi asisten.
2. **Build Bot:** Masuk ke folder ENAA_Nemesis, lalu jalankan perintah `dotnet build` untuk mengompilasi kode sumber C# menjadi file executable. Pastikan proses kompilasi selesai tanpa error sebelum melanjutkan ke tahap berikutnya.
3. **Menjalankan Bot:** Jalankan bot menggunakan perintah `dotnet run` pada direktori ENAA_Nemesis. Setelah berhasil dijalankan, bot akan aktif dan siap didaftarkan ke dalam arena. Pada tahap ini method `Run()` dijalankan untuk mengatur warna bot dan menginisialisasi loop utama, namun bot belum bergerak maupun menembak karena pertempuran belum dimulai.
4. **Memulai Pertempuran:** Pada game engine, pilih menu Battle lalu Mulai Pertarungan, kemudian tambahkan ENAA_Nemesis bersama bot musuh dan tekan Mulai. Setelah pertandingan dimulai, mesin permainan akan mengatur seluruh siklus giliran secara otomatis serta memanggil event handler seperti `OnScannedBot()`, `OnHitByBullet()`, dan `OnBotDeath()` setiap kali kondisi terkait terjadi hingga pertempuran selesai.
5. **Akhir Pertempuran:** Permainan berakhir ketika waktu habis atau kondisi lain terpenuhi, dan program menghentikan loop.

2.2.4 Kesimpulan

Algoritma greedy merupakan metode sederhana namun efektif untuk menyelesaikan masalah optimasi dengan selalu mengambil keputusan terbaik pada kondisi saat ini tanpa mempertimbangkan dampak jangka panjang. Pada ENAA_Nemesis, pendekatan ini diterapkan melalui beberapa mekanisme utama, yaitu mengunci musuh pertama yang terdeteksi radar tanpa mengevaluasi seluruh arena, menentukan firepower berdasarkan jarak target saat itu, menembak ke posisi perkiraan musuh menggunakan data pergerakan terbaru, serta mempertahankan radar lock agar informasi target terus diperbarui setiap giliran. Seluruh keputusan dilakukan secara mandiri di setiap iterasi tanpa perencanaan strategi jangka panjang.

Meskipun pendekatan ini cukup efektif untuk meningkatkan akurasi tembakan dan memaksimalkan damage terhadap satu target, terdapat beberapa keterbatasan. Bot belum mempertimbangkan energi lawan saat memilih target, tidak melakukan evaluasi ulang apabila target terkunci ternyata bukan ancaman utama, dan belum memiliki strategi khusus untuk menghadapi musuh yang bergerak secara adaptif atau aktif menghindar. Dalam situasi rumit dengan sejumlah bot yang bergerak secara dinamis, perencanaan jangka panjang seperti evaluasi multi-target atau perencanaan jalur mungkin akan memberikan kinerja yang lebih baik.

BAB III

APLIKASI STRATEGI GREEDY

Strategi greedy dalam Robocode Tank Royale diterapkan untuk meningkatkan keputusan bot pada setiap turn pertempuran, meliputi pemilihan sasaran, penentuan Firepower, penargetan tembakan, dan pengelolaan radar. Pada bab ini terdiri dari empat komponen yaitu: Proses Mapping Persoalan Bot Robocode Tank Royale ke Elemen Algoritma Greedy, Eksplorasi Empat Alternatif Solusi Greedy, Analisis Efisiensi dan Efektivitas, serta Strategi Greedy yang Dipilih.

3.1 Proses Mapping Persoalan ke Elemen Greedy

Proses Mapping merupakan langkah pertama yang dilakukan oleh bot untuk mengenali situasi di arena pertempuran dan menetapkan prioritas keputusan yang diambil di setiap turn. Proses ini dilakukan saat inisialisasi bot di method Run() dan penilaian kondisi di OnScannedBot() dari kelas ENAA_Nemesis. Persoalan Robocode Tank Royale dipetakan ke elemen-elemen algoritma Greedy sebagai berikut:

Himpunan Kandidat (C)	Himpunan kandidat adalah semua aksi yang dapat dilakukan bot dalam satu turn, yaitu bergerak maju atau mundur, berbelok, memutar gun, memutar radar, menembak dengan berbagai firepower (0.1 - 3.0), atau diam.
Himpunan Solusi (S)	Himpunan Solusi Adalah subset aksi kandidat yang membentuk strategi lengkap selama satu pertempuran, yaitu sequence keputusan per turn selama bot masih aktif.
Fungsi Solusi	Fungsi Solusi Adalah sekumpulan keputusan yang menghasilkan bot mampu bertahan hidup hingga akhir dan atau memperoleh skor akumulasi tertinggi di antara semua bot yang dipertandingkan.
Fungsi Seleksi	Fungsi seleksi Adalah memilih aksi yang memaksimalkan keuntungan lokal pada turn saat ini yaitu tembak dengan firepower optimal jika gun sudah terarah ke musuh, lalu kunci

	target berdasarkan heuristik yang dipilih, dan rotasi radar secara kontinu.
Fungsi Kelayakan	Fungsi kelayakan Adalah Bot dinilai jika energi yang dimiliki mencukupi untuk menembak, gun tidak panas ($\text{GunHeat} == 0$), sudut gun terhadap target dalam batas toleransi, dan bot tidak akan menabrak dinding pada turn berikutnya.
Fungsi Objektif	Fungsi objektif Adalah memaksimalkan total skor akhir yaitu skor = Bullet Damage + Bullet Damage Bonus + Survival Score + Last Survival Bonus + Ram Damage + Ram Damage Bonus. Agar mendapatkan skor sebanyak banyaknya

3.2 Eksplorasi Empat Alternatif Solusi Greedy

3.2.1 Bot Utama

ENAA_Nemesis: Circular Movement + Target Locking + Predictive Firing

Heuristik: Mengunci musuh pertama yang terdeteksi, perkiraan posisinya berdasarkan kecepatan dan arah, lalu tembak dengan firepower adaptif berdasarkan jarak sambil bergerak melingkar terus-menerus.

ENAA_Nemesis mengimplementasikan tiga komponen greedy sekaligus yang saling melengkapi:

1. **Circular Movement:** Bot bergerak dengan $\text{SetTurnLeft}(1.7) + \text{SetForward}(10.000)$ secara bersamaan, gerakan ini menciptakan pola gerak melingkar yang membuat bot sulit ditembak karena selalu bergerak. Gerakan melingkar dipilih karena gerakan ini adalah gerakan yang paling minim terkena peluru dari musuh, karena ketika musuh menembak seringkali peluru tidak terkena body bot karena bot sudah lebih dahulu bergerak melingkar, sehingga peluru hanya mengenai tempat kosong.
2. **Target Locking (First-Lock Greedy):** Ketika radar mendeteksi musuh pertama kali, bot langsung mengunci ($_lockedBotId$) dan tidak berpindah ke target yang lain selama musuh tersebut masih hidup. Keputusan greedy ini mengutamakan konsistensi damage ke satu target daripada menyebar ke

banyak musuh. Konsistensi ini lebih efektif dalam perolehan skor dibandingkan harus terus menerus memutar radar untuk mencari musuh.

3. **Predictive Firing:** Bot menghitung posisi perkiraan musuh menggunakan rumus: $\text{predictedX} = X + \sin(\text{direction}) \times \text{speed} \times \text{ticks}$, di mana $\text{ticks} = \text{distance} / \text{bulletSpeed}$. Gun diarahkan ke posisi perkiraan musuh berada, bukan posisi saat ini. Hal ini berguna untuk mencegah bot menembak tempat kosong karena musuh sudah meninggalkan letak prediksi sebelum peluru sampai di lokasi.
4. **Adaptive Firepower Greedy:** Firepower dipilih secara greedy berdasarkan jarak, jika jarak $< 150\text{px}$, firepower yang dikeluarkan sebesar 3.0 (damage maksimal). Jika jarak $150\text{--}400\text{px}$, firepower yang dikeluarkan adalah 2.5. dan jika jarak $> 400\text{px}$, firepower yang dikeluarkan sebesar 2.0. hal ini memaksimalkan kekuatan peluru yang besar, karena semakin dekat bot dengan musuh, akurasi tembaknya semakin besar, sehingga perbesaran firepower dibutuhkan saat letak musuh dekat.
5. **Radar Infinity Lock:** Radar diputar sebesar $\text{radarBearing} \times 1.5$ setiap scan, radar ini memastikan target selalu dalam scan arc.

Jika target hilang selama lebih dari `LOST_TARGET_TIMEOUT` (10 turn), bot akan mereset target dan mencari musuh baru dengan memutar radar 360° .

Komponen skor yang dioptimalkan: Bullet Damage (damage konsisten ke satu target), Bullet Damage Bonus (fokus membunuh satu musuh), Survival Score (circular movement meminimalkan kemungkinan tertembak).

3.2.2 Bot Alternatif 1

ENAA_Poseidon: Spin-and-Shoot Greedy

Heuristik: Merotasi body terus-menerus dengan kecepatan penuh dan tembak firepower maksimal setiap kali radar mendeteksi musuh apapun, tanpa membedakan target.

ENAA_Poseidon mengimplementasikan strategi greedy paling sederhana, yaitu berputar terus-menerus (`SetTurnLeft(10.000)`) sambil bergerak maju dengan

kecepatan 5. Premis greedy-nya adalah dengan selalu berputar, radar yang mengikuti body akan secara kontinu menlacak seluruh arena, sehingga peluang mendeteksi musuh sangat tinggi. Gerakan melingkar ini juga memberikan keuntungan untuk tidak mudah diprediksi musuh. Sama dengan Nemesis, tujuan gerak melingkar pada Poseidon agar ketika musuh menembakkan peluru ke letak prediksi bot, bot sudah terlebih dahulu meninggalkan posisi perkiraan sehingga peluru hanya mengenai tempat kosong.

Saat OnScannedBot() dipanggil, bot langsung menembak dengan firepower 3.0 (maksimal) tanpa evaluasi jarak atau arah gun. Ini adalah keputusan greedy murni, yaitu memanfaatkan setiap deteksi musuh semaksimal mungkin.

Saat terkena peluru, bot melakukan dodge acak (maju atau mundur sejauh 50–150 piksel). Hal ini bertujuan agar gerak bot tidak mudah di perkirakan oleh musuh, jadi ketika posisinya saat itu sudah diketahui, maka dia akan mengubah posisinya dengan bergerak maju atau mundur agar pola pergerakannya berbeda dari sebelumnya. Selain itu, saat menabrak bot musuh dengan jarak bearing $\pm 10^\circ$, bot langsung menembak 3.0 untuk memanfaatkan kesempatan jarak dekat, karena semakin dekat posisi bot dengan musuh, semakin besar kemungkinan akurasi penembakan.

Komponen skor yang dioptimalkan: Bullet Damage (volume tembakan tinggi), Ram Damage (sering menabrak karena berputar cepat), Survival (dodge acak menghindari peluru).

Kelemahan: Tidak ada perkiraan posisi musuh, sehingga tingkat akurasi mendeteksi letak musuh rendah. Gun tidak diputar independen dari body, hal ini menyebabkan banyak peluru meleset.

3.2.3 Bot Alternatif 2

ENAA Predator: Zigzag Movement + Opportunistic Firing

Heuristik: Bergerak zigzag dengan membalik arah setiap 25 tick dan membelok secara acak, sambil menembak setiap kali gun terarah ke musuh dan dingin. Hal ini

membuat bot mudah diprediksi dengan musuh karena gerakannya acak dan membuat musuh seringkali menembakkan peluru dengan meleset (tidak akurat)

ENAA_Predator mengimplementasikan strategi greedy berbasis pergerakan unpredictable. Bot bergerak dengan kecepatan maksimum (8 unit/turn) dan membalik arah setiap 25 tick ($\text{moveDir} *= -1$). Selain itu, TurnRate diatur secara acak ($5-25^\circ$) dengan arah yang acak juga, membuat trajektori bot sangat sulit diperkirakan musuh.

Keputusan greedy dalam pemilihan Gerakan Adalah bot selalu bergerak dengan kecepatan tertinggi yang memungkinkan untuk memaksimalkan coverage area dan meminimalkan waktu berada di satu titik serta mengurangi kemungkinan tertembak. Bot ini sangat lincah dan susah ditembak oleh bot yang bergerak secara terstruktur seperti bot yang bergerak melingkar dari jauh.

Untuk penembakan, bot mengoreksi GunTurnRate langsung ke bearing musuh saat scan, lalu menembak segera setelah gun dingin ($\text{GunHeat} == 0$). Firepower yang digunakan dipilih berdasarkan energi. Jika energi < 10 , firepower yang dikeluarkan sebesar 0.5. dan jika energi < 20 , firepower yang dikeluarkan sebesar 1.5, dan lainnya sebesar 3.0.

Saat terkena peluru, bot membalik arah dan memutar 90° secara acak untuk menghindari tembakan berikutnya dari musuh. Gunanya adalah mengubah pola gerakan agar tidak mudah diperkirakan oleh bot musuh. Selain itu, saat menabrak dinding, bot membalik arah dan reset tick counter.

Komponen skor yang dioptimalkan: Bullet Damage (menembak setiap kesempatan), Survival Score (gerakan zigzag unpredictable menyulitkan musuh mengincar), Ram Damage (kecepatan tinggi meningkatkan peluang tabrakan yang menguntungkan).

Kelemahan: Pergerakan acak tidak selalu optimal dalam mendekat ke musuh. Tidak ada target locking, sehingga damage tidak terfokus.

3.2.4 Bot Alternatif 3

ENAA_Gaia: Wall-Hugging + Predictive Firing Greedy

Heuristik: Bergerak menyusuri tepi arena (wall-hugging) untuk membatasi sudut serangan musuh, sambil menembak dengan perkiraan posisi musuh dan firepower adaptif berdasarkan energi.

ENAA_Gaia mengimplementasikan strategi greedy wall-hugging: bot selalu bergerak sepanjang sisi-sisi arena (`TurnRight(Direction % 90)` diikuti `Forward(maxDimension)`). Premis greedy-nya adalah dengan selalu berada di tepi arena sehingga musuh hanya dapat menyerang dari satu sisi, mengurangi exposure. Bot ini menggunakan strategi bergerak yang mencari aman karena alih alih mendekatkan diri dengan musuh, dia menembakkan musuh dari jauh tanpa berusaha mendekati musuh.

Saat mendeteksi musuh (`OnScannedBot`), Gaia melakukan:

6. **Kalkulasi Perkiraan Posisi:** $\text{predX} = X + \sin(\text{direction} \times \pi/180) \times \text{speed} \times (\text{distance}/\text{bulletSpeed})$. Ini mirip dengan Nemesis, namun Gaia menggunakan firepower yang berbeda.
7. **Adaptive Firepower Berbasis Energi:** Greedy dalam pemilihan daya tembak berdasarkan energi saat ini. Jika energi < 10 , maka firepower yang dikeluarkan sebesar 0.5 (hemat energi). Jika energi < 20 , firepower yang dikeluarkan sebesar 1.5. dan jika energi ≥ 20 , firepower yang dikeluarkan sebesar 3.0 (maksimal).
8. **Rescan dengan peek:** Setelah menembak, jika `flag peek == true`, bot melakukan `Rescan()` untuk langsung memindai ulang tanpa menunggu turn berikutnya.

Saat terkena peluru, bot melakukan dodge dengan bergerak maju 30–80 piksel, belok kanan 90° , lalu maju lagi 50–150 piksel. Tujuannya adalah membentuk pola L-shape dodge yang cukup efektif. Flag dodging mencegah re-entry dodge yang bertumpuk yang mengakibatkan bot stuck di pojok arena.

Komponen skor yang dioptimalkan: Bullet Damage (perkiraan posisi meningkatkan akurasi), Survival Score (wall-hugging mengurangi exposure, dodge

menghindari peluru masuk), Last Survival Bonus (cenderung bertahan lama karena gerakan terprediksi tapi aman).

Kelemahan: Gerakan wall-hugging sangat terprediksi oleh musuh yang cerdas. Serta tidak ada target locking, sehingga gun selalu diarahkan ke musuh terakhir yang terdeteksi.

3.3 Analisis Efisiensi dan Efektivitas

Untuk mengevaluasi keempat alternatif solusi greedy, dua aspek utama dijadikan acuan penilaian adalah efisiensi, yang mencakup kompleksitas komputasi dan kecepatan pengambilan keputusan per turn, serta efektivitas, yang mencerminkan kemampuan bot dalam memaksimalkan total skor akhir pertempuran. Dari evaluasi keempat strategi sebelumnya, sisi efisiensi komputasi dan efektivitas dalam memperoleh skor:

Bot	Kompleksitas	Efisiensi	Efektivitas	Kelebihan	Kelemahan
Nemesis	$O(n)$	Tinggi	Sangat Tinggi	Damage terfokus, akurasi tinggi	Risiko deadlock jika target kabur
Poseidon	$O(1)$	Sangat Tinggi	Rendah - Sedang	Sederhana, volume tembakan tinggi	Akurasi sangat rendah, boros energi
Predator	$O(n)$	Tinggi	Sedang	Unpredictable, susah ditembak	Damage tidak terfokus
Gaia	$O(n)$	Tinggi	Sedang–Tinggi	Akurasi baik, bertahan lama	Gerakan terprediksi

3.4 Strategi Greedy yang Dipilih

Berdasarkan analisis efisiensi dan efektivitas di atas, ENAA_Nemesis dipilih sebagai bot utama dengan alasan-alasan berikut:

1. **Damage terfokus dan konsisten:** Dengan mengunci satu target hingga mati, Nemesis mengakumulasi Bullet Damage dan Bullet Damage Bonus secara efektif. Damage bonus 20% dihasilkan saat membunuh musuh jauh lebih mudah diraih jika damage diarahkan ke satu bot. Berbeda dengan bot lain yang terus menerus melacar musuh menggunakan radar, strategi greedy target locking ini lebih efektif karena dia fokus menembak 1 musuh sampai mati dan mendapatkan Damage bonus 20% lalu mencari bot lain untuk dijadikan target selanjutnya, jadi dia mendapatkan skor optimal hasil dari menembak, dan dia mendapatkan skor bonus karena sudah membunuh musuh.
2. **Perkiraan posisi meningkatkan akurasi:** Dibandingkan Poseidon yang menembak tanpa perkiraan, Nemesis menghitung posisi musuh beberapa turn ke depan sehingga hit rate jauh lebih tinggi. Ini jelas lebih efektif karena seringkali bot lain menembak setelah musuh terkena radar, tanpa perhitungan tambahan sehingga peluru meleset dan hanya mengenai tempat kosong karena musuh sudah meninggalkan letak perkiraan. Nemesis melakukan perhitungan untuk memperkirakan posisi musuh saat peluru mencapai target, karena musuh kemungkinan sudah berpindah dari posisi awalnya. Sehingga kemungkinan peluru meleset tepat sasaran lebih besar.
3. **Circular movement mengurangi kemungkinan tertembak:** Pergerakan melingkar yang konstan membuat Nemesis sulit ditembak, berkontribusi pada Survival Score yang lebih tinggi. Gerakan melingkar dipilih karena gerakan ini lebih sulit di prediksi. Jika menggunakan gerakan acak bot tidak konsisten dibandingkan gerak melingkar yang konsisten. sehingga bot dapat menghindari serangan dengan lebih efektif sekaligus menjaga posisi tetap terkendali. Namun jika dibandingkan dengan gerakan menempel pada dinding, gerakan melingkar membuat pergerakan bot lebih fleksibel dan tidak terlalu mudah diprediksi. Sehingga gerakan melingkar yang dipilih karena dia konsisten namun fleksibel.
4. **Adaptive firepower memaksimalkan damage per energi:** Dengan menembak lebih keras saat dekat dan lebih hemat saat jauh, Nemesis mengoptimalkan rasio damage/energi yang dikeluarkan. Hal ini karena

logikanya jika semakin dekat bot dengan musuh, maka semakin besar kemungkinan akurasi bot menembak musuh dengan tepat, sehingga kesempatan jika bot berdekatan dengan musuh harus dimanfaatkan dengan baik dengan cara menembak dengan keras ketika dekat dengan musuh. Begitupula jika musuh berjarak jauh, maka akurasi ketepatan penembakan akan semakin kecil, sehingga peluru yang dikeluarkan lebih baik kecil saja untuk menghemat energi.

5. **Radar Infinity Lock:** Teknik lock radar memastikan musuh yang diincar selalu terpantau tanpa kehilangan tracking. Ini adalah fitur yang tidak didapatkan dari bot alternatif. Teknik Radar Infinity Lock membuat radar terus mengikuti gerak musuh yang sedang ditargetkan sehingga tracking berlangsung secara konsisten. Dengan cara ini, posisi musuh dapat diperbarui secara terus-menerus dan akurasi prediksi sehingga serangan menjadi lebih baik. Berbeda dengan bot lain yang mengandalkan sapuan radar akibat rotasi body, teknik tersebut berisiko kehilangan tracking ketika musuh bergerak cepat atau berada di luar area sapuan radar.

Kombinasi kelima komponen greedy ini menjadikan Nemesis strategi yang paling komprehensif dan konsisten dalam memaksimalkan skor total pertempuran.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

Bab ini menjelaskan secara rinci penerapan algoritma greedy pada bot ENAA_Nemesis, termasuk pseudocode, arsitektur keputusan per turn, serta pengujian bot dalam berbagai situasi pertempuran. Melalui penelitian ini, akan dieksplorasi seberapa efektif strategi greedy yang digunakan dalam mencapai performa optimal, dan mengidentifikasi situasi-situasi di mana pendekatan tersebut menunjukkan keterbatasan.

4.1 Implementasi Algoritma Greedy

Pengembangan ENAA_Nemesis didasarkan pada penerapan strategi greedy yang berfokus pada pengambilan keputusan lokal optimal di setiap turn pertempuran. Tujuannya adalah memaksimalkan hasil skor akhir dengan cara seefisien mungkin, melalui mekanisme target locking, adaptive firepower, predictive firing, dan radar infinity lock yang dikerjakan secara independen setiap turn.

4.1.1 Prinsip Desain ENAA_Nemesis

ENAA_Nemesis dibuat di atas lima pilar keputusan greedy yang saling melengkapi:

- **Circular Movement:** SetTurnLeft(1.7) + SetForward(10.000) dieksekusi bersamaan setiap turn, greedy ini menghasilkan pola gerak melingkar yang konsisten.
- **First-Lock Target Selection:** Setelah radar mendeteksi musuh sejak pertama kali, bot menyimpan ID musuh tersebut (`_lockedBotId`) dan mengabaikan semua bot lain sampai target yang dikunci tersebut mati.
- **Radar Infinity Lock:** Radar diputar sebesar $1.5 \times \text{radarBearingTo}(\text{target})$ jadi setiap kali target terdeteksi, greedy ini memastikan target selalu dalam scan arc.
- **Predictive Firing:** Posisi target diperkirakan berdasarkan kecepatan dan arah saat ini, gun diarahkan ke posisi perkiraan target, bukan posisi saat ini.

- **Adaptive Firepower:** Firepower dipilih secara greedy berdasarkan jarak. analoginya adalah jika musuh dekat maka firepower yang dikeluarkan kuat, jika musuh jauh maka firepower yang dikeluarkan mode hemat energi.

4.1.2 Pseudocode Detail ENAA_Nemesis

Berikut pseudocode detail untuk ENAA_Nemesis:

```
using System;
using System.Drawing;
using Robocode.TankRoyale.BotApi;
using Robocode.TankRoyale.BotApi.Events;
using System.IO;
using Microsoft.Extensions.Configuration;

public class ENAA_Nemesis : Bot
{
    // penanda apakah bot punya target atau belum
    private bool _hasTarget = false;

    // menyimpan posisi terakhir target
    private double _targetX;
    private double _targetY;

    // menyimpan id musuh yang sedang di-lock
    private int _lockedBotId = -1;

    // menyimpan turn terakhir musuh yang terdeteksi radar
    private int _lastScannedTurn = -1;

    // batas maksimal target hilang sebelum dianggap lost
    private const int LOST_TARGET_TIMEOUT = 10;

    static void Main(string[] args)
    {
        // load konfigurasi bot dari file json
        var builder = new ConfigurationBuilder()
            .SetBasePath(Directory.GetCurrentDirectory())
            .AddJsonFile("ENAA_Nemesis.json");

        var config = builder.Build();

        // ambil info bot dari konfigurasi
        var botInfo = BotInfo.FromConfiguration(config);

        // jalankan bot
        new ENAA_Nemesis(botInfo).Start();
    }

    // constructor bot
    private ENAA_Nemesis(BotInfo botInfo) : base(botInfo) { }

    public override void Run()
    {
        // mengatur warna tiap bagian bot
        BodyColor = Color.FromArgb(0xFF, 0xC2, 0x18, 0x5B);
        TurretColor = Color.FromArgb(0xFF, 0x8B, 0x00, 0x38);
        RadarColor = Color.FromArgb(0xFF, 0xF0, 0x62, 0x92);
        ScanColor = Color.FromArgb(0xFF, 0xFF, 0x14, 0x93);
        BulletColor = Color.FromArgb(0xFF, 0x3D, 0x0B, 0x1F);
        TracksColor = Color.FromArgb(0xFF, 0x88, 0x00, 0x33);
    }
}
```

```

GunColor    = Color.FromArgb(0xFF, 0xAD, 0x14, 0x57);

// loop utama bot selama game masih berjalan
while (IsRunning)
{
    // bot belok terus supaya gerak melingkar
    SetTurnLeft(1.7);

    // batas kecepatan maksimum
    MaxSpeed = 6;

    // maju terus supaya nonstop jalan
    SetForward(10_000);

    // mengecek apakah target udah terlalu lama ga ke-scan
    bool targetLost = _hasTarget &&
        (_lastScannedTurn < 0 ||
         TurnNumber - _lastScannedTurn > LOST_TARGET_TIMEOUT);

    // kalau target hilang
    if (targetLost)
    {
        // reset semua data target
        _hasTarget = false;
        _lockedBotId = -1;
        _lastScannedTurn = -1;

        // radar muter full nyari target baru
        SetTurnRadarLeft(360);
    }

    // kalau belum punya target dari awal
    else if (!_hasTarget)
    {
        // radar muter nyari musuh
        SetTurnRadarLeft(360);
    }

    // eksekusi semua perintah gerakan
    Go();
}

public override void OnScannedBot(ScannedBotEvent e)
{
    // kalau belum punya target, lock musuh pertama yang ketemu
    if (!_hasTarget)
    {
        _hasTarget = true;
        _lockedBotId = e.ScannedBotId;
    }

    // abaikan musuh lain selain yang lagi di-lock
    if (e.ScannedBotId != _lockedBotId) return;

    // update turn terakhir target terdeteksi
    _lastScannedTurn = TurnNumber;

    // update posisi target
    _targetX = e.X;
    _targetY = e.Y;

    // hitung arah radar ke target
    double radarBearing = RadarBearingTo(e.X, e.Y);

```

```

// radar diputar lebih besar biar tracking ga gampang lepas
SetTurnRadarLeft(radarBearing * 1.5);

// hitung power peluru berdasarkan jarak musuh
double firePower = CalcFirePower(DistanceTo(e.X, e.Y));

// rumus kecepatan peluru
double bulletSpeed = 20 - (3 * firePower);

// estimasi waktu peluru sampai ke target
double ticks = DistanceTo(e.X, e.Y) / bulletSpeed;

// prediksi posisi X musuh saat peluru sampai
double predictedX = e.X + Math.Sin(ToRad(e.Direction)) * e.Speed * ticks;

// prediksi posisi Y musuh saat peluru sampai
double predictedY = e.Y + Math.Cos(ToRad(e.Direction)) * e.Speed * ticks;

// arah turret ke posisi prediksi musuh
double gunBearing = GunBearingTo(predictedX, predictedY);

// putar turret ke target prediksi
SetTurnGunLeft(gunBearing);

// nembak kalau arah turret udah cukup akurat dan gun ga panas
if (Math.Abs(gunBearing) <= 15 && GunHeat == 0)
    Fire(firePower);
}

public override void OnBotDeath(BotDeathEvent e)
{
    // kalau target yang di-lock mati
    if (e.VictimId == _lockedBotId)
    {
        // reset data target
        _hasTarget = false;
        _lockedBotId = -1;
        _lastScannedTurn = -1;
    }
}

public override void OnHitByBullet(HitByBulletEvent e)
{
    // mundur dikit pas kena peluru buat menghindar
    SetBack(30);
}

public override void OnHitWall(HitWallEvent e)
{
    // mundur kalau nabrak tembok
    SetBack(40);
}

public override void OnHitBot(HitBotEvent e)
{
    // kalau ditabrak bot lain, belok
    if (e.IsRammed) TurnLeft(10);
}

private double CalcFirePower(double distance)
{
    // kalau musuh dekat, pakai damage besar
    if (distance < 150) return 3;
}

```

```

        // jarak menengah
        if (distance < 400) return 2.5;

        // jarak jauh pakai power supaya lebih hemat
        return 2;
    }

    private double ToRad(double degrees)
    {
        // konversi derajat ke radian
        return degrees * Math.PI / 180.0;
    }
}

```

4.1.3 Penjelasan Komponen Kunci Implementasi ENAA_Nemesis

a. Run(): Method ini merupakan bagian pertama yang berjalan saat bot diaktifkan. Isinya hanya ada dua hal:

1. Mengatur warna bot melalui SetColors() sebagai inisialisasi visual, kemudian langsung masuk ke loop WHILE isRunning yang berjalan tanpa henti selama bot aktif. Di dalam loop tersebut, setiap turn bot selalu mengeksekusi SetTurnLeft(1.7) dan SetForward(10000) dengan MaxSpeed = 6. Kombinasi ini yang menghasilkan pola gerakan melingkar konstan tanpa bot perlu berhenti.

2. Memeriksa nilai hasTarget dan lastScannedTurn untuk menentukan apakah target masih ada atau tidak. Jika TurnNumber - lastScannedTurn sudah melewati LOST_TARGET_TIMEOUT, bot menganggap target hilang dan mereset semua state. Jika memang belum punya target sama sekali, radar langsung diputar penuh dengan SetTurnRadarLeft(360) untuk mencari musuh. Setelah semua perintah disiapkan dalam satu turn, Go() dipanggil untuk mengirim semuanya ke game engine sekaligus dalam satu waktu.

b. CalcFirePower(): Fungsi CalcFirePower(distance) menentukan seberapa kuat tembakan berdasarkan jarak ke musuh saat itu. Jika jarak kurang dari 150 piksel, fungsi mengembalikan firepower maksimum 3.0 karena musuh dekat dan peluru hampir pasti mengenai target. Jika jarak antara 150 hingga 400 piksel, firepower diturunkan ke 2.5 sebagai keseimbangan antara damage dan kecepatan peluru. Jika jarak lebih dari 400 piksel, firepower diturunkan lagi ke 2.0 supaya peluru bergerak lebih cepat ($\text{bulletSpeed} = 20 - (3 \times \text{firePower})$) dan energi tidak terbuang sia-sia untuk tembakan yang sulit mengenai target dari jarak jauh. Logika ini yang

membuat bot tidak asal tembak maksimum terus menerus, melainkan selalu menyesuaikan kekuatan tembakan dengan kondisi jarak di arena saat itu.

c. OnScannedBot(): Setiap kali radar mendeteksi musuh yang dikunci melalui event OnScannedBot(e), method ini langsung menjalankan empat hal secara berurutan.

1. Radar diputar dengan SetTurnRadarLeft(radarBearing $\times$ 1.5). Angka 1.5 sengaja dibuat lebih dari yang dibutuhkan supaya musuh selalu tertangkap di tengah scan arc dan tidak meleset di pinggir.
2. Bot mengukur jarak ke musuh lalu memanggil CalcFirePower(distance) untuk mendapat nilai firepower yang paling efisien, yang hasilnya langsung dipakai menghitung kecepatan peluru.
3. Karena peluru butuh waktu sampai ke target, bot menghitung ticks = distance / bulletSpeed lalu memperkirakan posisi musuh setelah sekian turn menggunakan e.Speed dan e.Direction dari data scan, menghasilkan predictedX dan predictedY.
4. Gun diarahkan ke titik prediksi itu dengan SetTurnGunLeft(gunBearing), dan tembakan dilepas lewat Fire(firePower) hanya jika dua syarat terpenuhi sekaligus yaitu sudut meleset tidak lebih dari 15° dan meriam tidak sedang panas (GunHeat = 0). Semua keputusan menyerang bot terpusat di sini.

d. OnBotDeath(): Mereset semua hasTarget = false, lockedBotId = -1, dan lastScannedTurn = -1 jika musuh yang sedang dikunci mati. Hapus ID target, hapus catatan waktu scan, dan tandai bahwa bot belum punya target lagi. Setelah ini, di turn berikutnya Run() akan otomatis langsung memutar radar 360° untuk mencari musuh baru.

e. Mekanisme Target Timeout dalam Run(): Mekanisme target timeout dalam Run() berfungsi sebagai sistem pengaman agar bot tidak terjebak menunggu musuh yang sudah tidak ada. Setiap turn, bot membandingkan TurnNumber dengan lastScannedTurn. Jika selisihnya sudah melewati LOST_TARGET_TIMEOUT (10 turn), maka variabel targetLost bernilai true dan bot langsung mereset seluruh state target: hasTarget dikembalikan ke false, lockedBotId ke -1, dan lastScannedTurn ke -1. Setelah reset, SetTurnRadarLeft(360) dipanggil untuk memutar radar penuh mencari musuh baru. Tanpa mekanisme ini, bot bisa terus mengira masih punya

target padahal musuh sudah kabur atau mati tanpa terdeteksi, sehingga radar tidak pernah beralih ke mode scan penuh dan bot kehilangan waktu berharga untuk mengunci musuh berikutnya.

f. Logika Keputusan per Turn dalam Run():

1. **Prioritas Pergerakan Sebelum Evaluasi Kondisi:** Setiap turn, perintah SetTurnLeft(1.7), MaxSpeed = 6, dan SetForward(10000) selalu di-set pertama sebelum apapun diperiksa. Urutan ini disengaja dengan menempatkan perintah gerak di awal loop, bot dipastikan tidak pernah berdiri diam meskipun sedang memproses pengecekan target atau kondisi lainnya. Nilai SetForward(10000) yang sangat besar memastikan perintah maju tidak habis dalam satu turn, sehingga bot terus bergerak tanpa perlu di-set ulang setiap saat.
2. **Evaluasi Status Target:** Setelah perintah gerak di-set, bot memeriksa nilai hasTarget dan menghitung selisih TurnNumber - lastScannedTurn untuk menentukan apakah target masih valid. Jika target sudah timeout atau memang belum ada sama sekali, bot mengalihkan radar ke mode scan penuh dengan SetTurnRadarLeft(360). Pengecekan ini sengaja diletakkan setelah perintah gerak supaya apapun hasilnya, baik target masih ada maupun tidak, bot tetap sudah dalam kondisi bergerak dan tidak pernah diam menunggu hasil evaluasi.
3. **Pemisahan Tanggung Jawab Antara Run() dan OnScannedBot():** Keputusan menembak sama sekali tidak ada di Run(). Seluruh logika penembakan sepenuhnya dikendalikan oleh OnScannedBot() yang dipanggil otomatis oleh game engine setiap kali radar mendeteksi target yang dikunci. Pemisahan ini disengaja Run() hanya bertanggung jawab atas dua hal yaitu menjaga bot tetap bergerak dan mengelola status radar. Semua keputusan menyerang seperti menghitung firepower, memprediksi posisi musuh, mengarahkan gun, dan melepas tembakan sepenuhnya menjadi tanggung jawab OnScannedBot() sebagai event handler yang tepat untuk menangani hal tersebut.

4.2 Struktur Data yang Digunakan dalam Program Bot ENAA_Nemesis

Pemilihan struktur data ini bertujuan untuk mendukung efisiensi dan kesederhanaan logika greedy pada bot ENAA_Nemesis:

Variabel / Method	Tipe	Deskripsi
_hasTarget	bool	Flag apakah bot sudah memiliki target terkunci
_lockedBotId	int	ID musuh yang sedang dikunci (-1 jika belum ada target)
_targetX, _targetY	double	Koordinat terakhir musuh yang terkunci (dari scan terakhir)
_lastScannedTurn	int	Turn terakhir musuh terkunci berhasil discan oleh radar
LOST_TARGET_TIMEOUT	const int	Batas toleransi kehilangan target: 10 turn
CalcFirePower(dist)	double	Mengembalikan firepower greedy berdasarkan jarak ke target
ToRad(degrees)	double	Konversi derajat ke radian untuk kalkulasi trigonometri
Run()	override void	Loop utama: circular movement + radar scan management
OnScannedBot(e)	override void	Handler utama: target lock, radar lock, prediksi, tembak
OnBotDeath(e)	override void	Reset semua state target saat musuh terkunci mati
OnHitByBullet(e)	override void	Reaksi defensif: mundur 30 unit
OnHitWall(e)	override void	Reaksi dinding: mundur 40 unit

Tabel di atas menunjukkan seluruh variabel dan method yang digunakan ENAA_Nemesis. Bot hanya menyimpan informasi yang benar-benar diperlukan untuk membuat keputusan di turn itu. _hasTarget dan _lockedBotId adalah dua variabel paling penting, keduanya selalu dicek setiap turn untuk menentukan

apakah bot perlu scan ulang atau langsung menyerang. `_targetX`, `_targetY`, dan `_lastScannedTurn` nilainya selalu diperbarui setiap kali radar berhasil mendeteksi musuh, jadi data yang dipakai untuk menembak selalu yang paling baru. `LOST_TARGET_TIMEOUT` hanyalah angka batas jika musuh tidak terdeteksi lebih dari 10 turn, bot otomatis reset dan cari target baru.

Untuk method-nya, `Run()` dan `OnScannedBot()` adalah dua yang paling sering bekerja karena keduanya aktif setiap turn. `Run()` fokus mengurus gerak dan radar, `OnScannedBot()` fokus mengurus semua keputusan menyerang. Tiga event handler lainnya `OnBotDeath()`, `OnHitByBullet()`, `OnHitWall()`, dan `OnHitBot()` hanya aktif kalau kejadian tertentu memang terjadi di arena, jadi tidak selalu dipanggil setiap turn.

4.3 Analisis Solusi Algoritma Greedy Melalui Skenario Pengujian

Bagian ini membahas perancangan solusi algoritma greedy ENAA_Nemesis dengan menilai kinerjanya dalam berbagai situasi pertempuran. Tujuannya adalah untuk mengevaluasi seberapa efektif strategi yang digunakan dalam memaksimalkan skor akhir, dan untuk mengidentifikasi situasi di mana strategi greedy ini mungkin kurang efektif.

ENAA_Nemesis dibuat untuk mengambil keputusan terbaik secara lokal di setiap giliran berdasarkan informasi yang ada pada saat itu posisi lawan, jarak, kecepatan, dan arah pergerakan. Bot tidak menyimpan catatan pertempuran sebelumnya atau merencanakan langkah-langkah di masa mendatang. Analisis berikut akan menjelajahi bagaimana metode ini bekerja dalam berbagai kemungkinan situasi pertempuran yang dapat muncul di arena.

4.3.1 Skenario Pengujian

a) Skenario Pengujian 1 – ENAA_Nemesis vs ENAA_Poseidon (1 vs 1)

Skenario pertama adalah pengujian terhadap Bot ENAA_Nemesis secara head-to-head melawan ENAA_Poseidon, bot alternatif dengan strategi spin-and-shoot tanpa perkiraan. Pengujian ini bertujuan untuk melihat sejauh mana perbedaan antara

greedy dengan predictive firing melawan greedy tanpa prediksi sama sekali berpengaruh terhadap hasil pertempuran.

Kondisi Pengujian	ENAA_Nemesis melawan ENAA_Poseidon dalam mode 1 vs 1, 10 ronde.
Perilaku Nemesis	Radar langsung mengunci Poseidon pada scan pertama. Circular movement aktif sepanjang ronde membuat Nemesis bergerak melingkar mendekati Poseidon secara spiral. Predictive firing aktif setiap scan gun diarahkan ke posisi prediksi Poseidon berdasarkan kecepatan dan arah rotasinya.
Hasil	ENAA_Nemesis menang dengan total skor 1072 vs 406. Bullet Damage Nemesis mencapai 641 dengan Bullet Bonus 94, jauh di atas Poseidon yang hanya 256 Bullet Damage tanpa Bullet Bonus. Nemesis meraih 6 kemenangan ronde (1st) dan 2 ronde kedua dari 10 ronde.
Analisis	Poseidon yang menembak tanpa prediksi dan tanpa target locking kesulitan mengenai Nemesis yang terus bergerak melingkar. Sebaliknya, Nemesis secara konsisten mengenai Poseidon karena pola rotasi Poseidon yang konstan mudah diprediksi. Bullet Bonus sebesar 94 membuktikan Nemesis berhasil menghabisi Poseidon berkali-kali dalam satu target lock. Ram Damage Nemesis sebesar 36 juga menunjukkan circular movement secara natural sering membawa Nemesis menabrak Poseidon.
Kondisi Tidak Optimal	Pada 2 ronde di mana Nemesis tidak menang, Poseidon sempat mengenai Nemesis secara acak dengan firepower 3.0 di jarak dekat sebelum Nemesis sempat mengunci dan mengarahkan gun. Tembakan volume tinggi Poseidon meskipun tidak

	akurat tetapi berpotensi mengenai target jika jarak sangat dekat.
--	---

Hasil Battle Bot Nemesis VS Bot Poseidon

Nama	Energy	Total S...
ENAA_Poseidon 1.0 (1)	81.0	406
ENAA_Nemesis 1.0 (2)	83.5	1072

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	ENAA_Nemesis 1.0	1072	250	50	641	94	36	0	6	2	0
2	ENAA_Poseidon 1.0	406	100	20	256	0	30	0	2	6	0

Robocode Tank Royale
Build the best - destroy the rest!

Dari pengujian ini dapat disimpulkan bahwa predictive firing memberikan keunggulan yang sangat signifikan dalam duel 1 vs 1. Selisih skor hampir 3x lipat (1072 vs 406) membuktikan bahwa menembak ke posisi prediksi jauh lebih efektif dibanding menembak tanpa kalkulasi, terutama melawan bot yang bergerak dengan pola konstan seperti Poseidon. Bullet Bonus yang tinggi juga mengkonfirmasi bahwa target locking ke satu musuh hingga mati adalah keputusan greedy yang tepat untuk mengakumulasi bonus damage.

b) Skenario Pengujian 2 – ENAA_Nemesis vs ENAA_Predator (1 vs 1)

Skenario kedua adalah pengujian terhadap Bot ENAA_Nemesis melawan Bot ENAA_Predator, bot alternatif yang dirancang khusus untuk menjadi unpredictable melalui gerakan zigzag dengan perubahan arah acak setiap 25 tick. Pengujian ini bertujuan untuk menguji ketahanan predictive firing Nemesis menghadapi gerakan yang tidak beraturan.

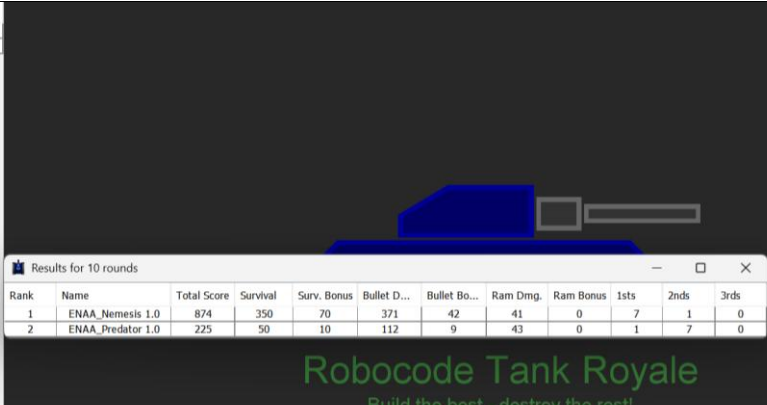
Kondisi Pengujian	ENAA_Nemesis melawan ENAA_Predator dalam mode 1 vs 1, 10 ronde.
Perilaku Nemesis	Radar mengunci Predator pada scan pertama. Circular movement aktif sepanjang ronde. Predictive firing

	terus memperbarui prediksi posisi Predator yang bergerak zigzag dengan arah dan kecepatan yang sering berubah setiap 25 tick. Setiap kali OnScannedBot dipanggil, prediksi diperbarui menggunakan data kecepatan dan arah Predator terbaru.
Hasil	ENAA_Nemesis menang dominan dengan total skor 874 vs 225. Bullet Damage Nemesis mencapai 371 dengan Bullet Bonus 42. Nemesis meraih 7 kemenangan ronde dari 10 ronde. Predator hanya meraih 1 kemenangan ronde.
Analisis	Meskipun Predator dirancang untuk menjadi unpredictable, dalam praktiknya pola zigzag dengan interval tetap 25 tick masih bisa diikuti oleh predictive firing Nemesis. Karena prediksi selalu diperbarui setiap scan menggunakan data terbaru, perubahan arah Predator tidak langsung membuat tembakan Nemesis meleset. gun Nemesis hanya perlu menyesuaikan arah di scan berikutnya. Bullet Bonus 42 membuktikan Nemesis berhasil menghabisi Predator berkali-kali secara konsisten. Ram Damage Predator yang mencapai 43 menunjukkan Predator sempat beberapa kali menabrak Nemesis, tapi tidak cukup untuk mengimbangi selisih Bullet Damage yang sangat besar.
Konisi Tidak Optimal	Pada 1 ronde di mana Predator menang, kemungkinan besar Predator melakukan perubahan arah mendadak tepat saat peluru Nemesis sudah berebeda posisi sehingga perkiraan posisi meleset secara signifikan. Selain itu Ram Damage Predator yang cukup besar menunjukkan ada momen di mana gerakan zigzag Predator secara tidak sengaja membawa Predator

	menabrak Nemesis dari sudut yang tidak terantisipasi oleh OnHitBot.
--	---

Hasil Battle Bot Nemesis VS Bot Predator

Name	Energy	Total S...
ENAA_Predator 1.0 (1)	97.0	225
ENAA_Nemesis 1.0 (2)	98.0	874



Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	ENAA_Nemesis 1.0	874	350	70	371	42	41	0	7	1	0
2	ENAA_Predator 1.0	225	50	10	112	9	43	0	1	7	0

Robocode Tank Royale

Build the best - destroy the rest!

Dari pengujian ini dapat disimpulkan bahwa gerakan zigzag Predator tidak cukup efektif mengecoh predictive firing Nemesis karena prediksi selalu diperbarui setiap turn berdasarkan data terbaru. Rasio kemenangan 7 dari 10 ronde menunjukkan bahwa Nemesis paling dominan melawan bot yang bergerak cepat namun dengan pola yang masih bisa dibaca secara lokal per turn.

c) Skenario Pengujian 3 – ENAA_Nemesis vs ENAA_Gaia (1 vs 1)

Skenario ketiga adalah pengujian terhadap Bot ENAA_Nemesis dengan Bot ENAA_Gaia, bot alternatif yang menggunakan strategi wall-hugging untuk meminimalkan sudut serangan musuh. Pengujian ini bertujuan untuk mengevaluasi seberapa efektif predictive firing Nemesis menghadapi bot yang pergerakannya terbatas di tepi arena namun memiliki dodge aktif.

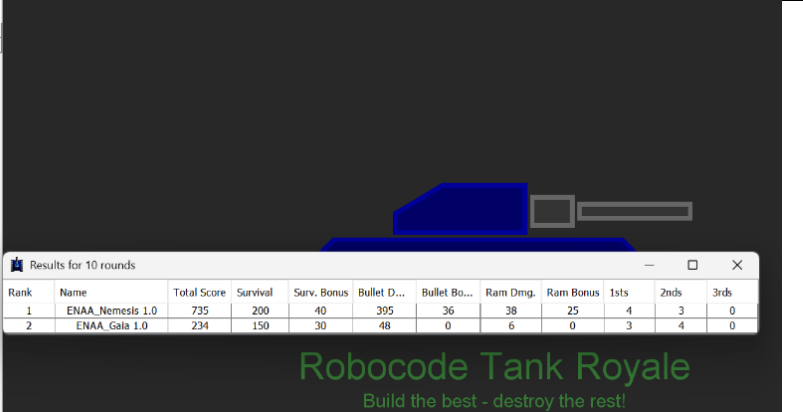
Kondisi Pengujian	ENAA_Nemesis melawan ENAA_Gaia dalam mode 1 vs 1, 10 ronde.
Perilaku Nemesis	Radar mengunci Gaia sejak scan pertama. Circular movement membawa Nemesis mendekati Gaia yang cenderung bergerak di tepi arena. Firepower adaptif menyesuaikan jarak — saat Gaia berada di tepi dan

	Nemesis masih jauh, firepower 2.0–2.5 digunakan, lalu naik ke 3.0 saat Nemesis sudah dekat. Predictive firing memperhitungkan pergerakan Gaia sepanjang sisi arena.
Hasil	ENAA_Nemesis menang dengan total skor 735 vs 234. Bullet Damage Nemesis mencapai 395 dengan Bullet Bonus 36. Nemesis meraih 4 kemenangan ronde (1st) dan 3 ronde kedua dari 10 ronde. Survival Nemesis 200 vs Gaia 150 menunjukkan Nemesis lebih sering bertahan lebih lama.
Analisis	Gaia yang bergerak menyusuri tepi arena memiliki pola gerakan yang cukup terprediksi — selalu mengikuti sisi arena dengan pola L-shape. Ini membuat predictive firing Nemesis cukup akurat karena arah dan kecepatan Gaia relatif konsisten. Namun selisih skor yang lebih kecil dibanding skenario lain menunjukkan Gaia cukup tangguh — wall-hugging movement meminimalkan sudut serangan dan dodge pola L-shape saat kena tembakan sempat beberapa kali membuat peluru Nemesis meleset dari prediksi. Bullet Bonus yang lebih kecil (36) dibanding skenario lain menandakan Gaia lebih sulit dihabisi dalam satu lock karena dodge aktifnya.
Kondisi Tidak Optimal	Pada 3 ronde di mana Nemesis tidak menang, Gaia kemungkinan berhasil memanfaatkan posisi sudut arena yang membatasi ruang gerak circular movement Nemesis. Ketika Nemesis terpaksa mundur karena menabrak dinding (OnHitWall → SetBack(40)), momentum gerakan melingkar terganggu dan Gaia sempat mendapatkan posisi tembak yang lebih menguntungkan. Prediksi posisi

	Gaia juga bisa meleset saat Gaia melakukan dodge L-shape mendadak setelah terkena peluru.
--	---

Hasil Battle Bot Nemesis VS Bot Gaia

Nama	Energy	Total S...
ENAA_Nemesis 1.0 (1)	99.25	735
ENAA_Gaia 1.0 (2)	97.0	234



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	ENAA_Nemesis 1.0	735	200	40	395	36	38	25	4	3	0
2	ENAA_Gaia 1.0	234	150	30	48	0	6	0	3	4	0

Robocode Tank Royale
Build the best - destroy the rest!

Dari pengujian ini dapat disimpulkan bahwa Gaia adalah lawan yang paling sulit di antara bot alternatif dalam duel 1 vs 1, terbukti dari jumlah ronde yang tidak dimenangkan Nemesis (3 ronde) yang lebih banyak dibanding lawan lainnya. Wall-hugging terbukti cukup efektif mengganggu ritme circular movement Nemesis, terutama di area sudut arena. Meski demikian, Nemesis tetap unggul secara keseluruhan karena predictive firing berhasil mengikuti pola L-shape Gaia di sebagian besar ronde.

d) Skenario Pengujian 4 – 4-Way Battle (Semua Bot ENAA)

Skenario terakhir menempatkan keempat bot ENAA dalam satu arena sekaligus untuk mensimulasikan kondisi pertempuran paling kompleks. Pengujian ini bertujuan untuk melihat bagaimana performa Nemesis saat harus menghadapi tiga musuh sekaligus dengan karakteristik yang berbeda-beda, sekaligus membuktikan konsistensi strategi greedy dalam kondisi multi-bot.

Kondisi Pengujian	Keempat bot ENAA bertanding sekaligus dalam satu arena, 10 ronde.
Perilaku Nemesis	Radar mengunci satu musuh pertama yang terdeteksi dan mempertahankan lock sepanjang ronde. Circular movement aktif. Bot lain yang tidak dikunci diabaikan sepenuhnya meskipun masuk jangkauan

	radar. Setiap kali target terkunci mati, OnBotDeath mereset state dan radar langsung mencari target baru.
Hasil	Peringkat 1: ENAA_Nemesis (1432), Peringkat 2: ENAA_Poseidon (768), Peringkat 3: ENAA_Gaia (432), Peringkat 4: ENAA_Predator (380). Nemesis meraih 4 kemenangan ronde dari 10 ronde dengan Bullet Damage 544, Bullet Bonus 67, dan Survival Score tertinggi sebesar 650.
Analisis	Survival Score Nemesis yang mencapai 650 , tertinggi di antara semua bot. Hal ini membuktikan circular movement dan predictive firing berhasil mempertahankan Nemesis hidup lebih lama di setiap ronde. Bullet Bonus 67 menunjukkan Nemesis berhasil menghabisi target terkunci berkali-kali dan langsung mengunci target baru melalui mekanisme OnBotDeath reset. Nemesis juga mendapat keuntungan pasif berupa Survival Score 50 poin setiap kali bot lain mati karena pertempuran antar sesama, bahkan tanpa harus terlibat langsung. Poseidon di peringkat 2 diuntungkan oleh volume tembakan tinggi dan Ram Damage terbesar (44), namun tanpa prediksi skornya jauh di bawah Nemesis. Gaia dan Predator kesulitan bersaing karena damage yang tidak terarah dan tidak ada mekanisme target locking.
Kondisi Tidak Optimal	Kelemahan paling nyata muncul di skenario ini Adalah Nemesis terkunci pada satu target sementara dua bot lain bebas menyerang dari arah manapun tanpa dipedulikan. Jika target pertama yang dikunci adalah bot yang paling tangguh dan berada di posisi jauh, Nemesis bisa kehilangan banyak energi dari serangan bot lain yang tidak dikunci. Selain itu tidak

	ada mekanisme untuk mendeteksi bot lain yang hampir mati di dekatnya sehingga kesempatan mendapat Bullet Damage Bonus dari bot yang hampir mati sering terlewat karena Nemesis tidak mau berpindah target meskipun target saat ini masih energi penuh.
--	--

Hasil Battle Keempat Bot ENAA

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	ENAA_Nemesis 1.0	1432	650	90	544	67	77	4	3	3	0
2	ENAA_Poseidon 1.0	768	300	60	336	22	44	5	3	0	1
3	ENAA_Gaia 1.0	432	350	0	64	0	18	0	0	3	1
4	ENAA_Predator 1.0	380	200	0	118	0	62	0	0	0	4

Dari keseluruhan pengujian 4 ini dapat disimpulkan bahwa strategi greedy ENAA_Nemesis terbukti konsisten unggul bahkan dalam kondisi pertempuran paling kompleks sekalipun. Selisih skor yang sangat besar antara Nemesis (1432) dan bot peringkat kedua Poseidon (768) hampir 2x lipat membuktikan bahwa kombinasi target locking, predictive firing, adaptive firepower, dan circular movement adalah strategi greedy yang paling efektif di antara keempat alternatif yang diuji. Survival Score tertinggi sekaligus Bullet Damage tertinggi menunjukkan Nemesis berhasil menyeimbangkan antara kemampuan bertahan dan kemampuan menyerang secara bersamaan dalam satu strategi yang kohesif.

4.3.2 Resume Keunggulan dan Kelemahan Bot ENAA_Nemesis

Bagian ini merangkum kekuatan dan kelemahan dari masing-masing bot berdasarkan hasil pengujian dan analisis yang telah dilakukan sebelumnya.

ENAA_Nemesis (Bot Utama)

Kekuatan:

- **Target Locking yang Konsisten:** Mengunci satu musuh hingga mati memastikan akumulasi Bullet Damage Bonus (20%) yang konsisten setiap ronde. Ini terbukti dari Bullet Bonus tertinggi di semua skenario pengujian dibanding bot lain.

- **Predictive Firing yang Efektif:** Menembak ke posisi prediksi musuh berdasarkan data kecepatan dan arah terbaru membuat hit rate jauh lebih tinggi dibanding menembak ke posisi saat ini, terutama efektif pada jarak menengah hingga jauh.
- **Circular Movement Defensif sekaligus Ofensif:** Gerakan melingkar konstan membuat Nemesis sulit dijadikan target sekaligus secara natural mendekati musuh secara spiral, memaksimalkan firepower saat jarak sudah dekat. Dibuktikan dengan Survival Score tertinggi di semua skenario.
- **Adaptive Firepower:** Pemilihan firepower berdasarkan jarak memastikan energi tidak terbuang sia-sia dan peluru selalu dalam kondisi optimal untuk mengenai target di jarak manapun.
- **Radar Infinity Lock:** Teknik $1.5\times$ bearing memastikan musuh tidak pernah lepas dari scan arc, sehingga data prediksi selalu diperbarui setiap turn tanpa jeda.

Kelemahan:

- **Tidak Ada Evaluasi Target:** Bot mengunci musuh pertama yang terdeteksi tanpa mempertimbangkan apakah itu pilihan terbaik. Di skenario multi-bot, ini bisa membuat Nemesis terfokus pada musuh yang paling tangguh sementara musuh yang hampir mati di dekatnya diabaikan sehingga kesempatan Damage bonus terlewatkan
- **Prediksi Linear:** Prediksi posisi musuh mengasumsikan musuh terus bergerak dengan kecepatan dan arah yang sama. Jika musuh melakukan manuver mendadak seperti menabrak dinding atau berbalik arah, prediksi bisa meleset signifikan.
- **Respon Defensif Terbatas:** OnHitByBullet hanya mengeksekusi SetBack(30) tanpa mempertimbangkan arah datangnya peluru. Tidak ada mekanisme untuk mendeteksi dari mana serangan datang atau menghindari secara lebih cerdas.

- **Rentan di Sudut Arena:** Circular movement bisa terganggu saat Nemesis terpaksa mundur karena menabrak dinding, memutus momentum gerakan melingkar yang menjadi kekuatan utamanya.

ENAA_Poseidon (Bot Alternatif 1)

- **Kekuatan:** Volume tembakan tinggi dengan firepower maksimum setiap saat, efektif di jarak sangat dekat karena kemungkinan mengenai target secara acak lebih tinggi. Ram Damage tertinggi di antara semua bot karena pergerakannya yang agresif dan cepat.
- **Kelemahan:** Tidak ada prediksi posisi dan tidak ada target locking, sehingga hit rate sangat rendah terutama di jarak menengah hingga jauh. Boros energi karena selalu menembak dengan firepower 3.0 tanpa mempertimbangkan efisiensi.

ENAA_Predator (Bot Alternatif 2)

- **Kekuatan:** Gerakan zigzag dengan perubahan arah acak dirancang untuk mengecoh predictive firing lawan. Kecepatan gerak maksimum membuat Predator sulit dikejar dalam jarak jauh.
- **Kelemahan:** Dalam praktiknya gerakan zigzag dengan interval tetap 25 tick masih bisa dibaca oleh predictive firing yang memperbarui data setiap scan. Tidak ada target locking membuat damage tersebar ke banyak musuh tanpa mengakumulasi Bullet Bonus dari satu target manapun. Peringkat terakhir di skenario 4-way dengan skor terendah (380) membuktikan strategi ini paling tidak efektif di antara keempat bot.

ENAA_Gaia (Bot Alternatif 3)

- **Kekuatan:** Wall-hugging meminimalkan sudut serangan musuh dan membuat posisi Gaia lebih terprediksi bagi dirinya sendiri. Dodge pola L-shape terbukti cukup efektif mengganggu predictive firing lawan dalam beberapa ronde.
- **Kelemahan:** Pola gerakan yang terbatas di tepi arena justru membuat Gaia mudah diprediksi oleh bot yang menggunakan predictive firing. Bullet

Damage sangat rendah karena tidak ada target locking dan firepower tidak dioptimalkan berdasarkan jarak secara konsisten.

Kesimpulannya, ENAA_Nemesis terbukti menjadi bot paling efektif di antara keempat bot yang dikembangkan, dengan strategi greedy yang berhasil mengoptimalkan empat aspek pertempuran sekaligus dalam satu implementasi yang sederhana namun kohesif. Hasil pengujian menunjukkan keunggulan yang konsisten di semua skenario, baik duel 1 vs 1 maupun pertempuran 4 bot. Meski begitu, keterbatasan pada evaluasi target dan respon defensif yang terbatas menjadi catatan penting yang perlu diperbaiki jika Nemesis ingin tampil lebih optimal menghadapi lawan yang lebih beragam dan tidak terpola dalam kompetisi sesungguhnya

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

ENAA_Nemesis menggunakan strategi greedy untuk membuat keputusan pertempuran di setiap turn, mencakup pemilihan target, pengelolaan radar, penentuan firepower, dan prediksi posisi musuh. Strategi ini memprioritaskan efisiensi damage dengan mengunci satu target hingga mati dan menembak ke posisi prediksi musuh berdasarkan kecepatan serta arah gerakannya, memastikan setiap peluru yang dilepas memiliki probabilitas mengenai target yang lebih tinggi dibanding tembakan langsung. Bot juga menerapkan adaptive firepower berdasarkan jarak maksimum saat dekat, dan lebih rendah saat jauh untuk menjaga efisiensi energi sepanjang pertempuran. Selain itu, circular movement yang berjalan konstan setiap turn membuat Nemesis sulit dijadikan target sekaligus secara natural mendekati musuh secara spiral.

Method Run() berfungsi sebagai loop utama yang berjalan terus selama bot aktif, menangani inisialisasi warna bot, pergerakan melingkar, pengelolaan status target, dan pengiriman perintah ke game engine melalui Go(). Seluruh logika menyerah difokuskan ke OnScannedBot() yang dipanggil otomatis oleh game engine setiap kali radar mendeteksi target terkunci, sehingga pemisahan tanggung jawab antar method terjaga dengan baik.

Secara keseluruhan, strategi greedy yang diimplementasikan pada ENAA_Nemesis sangat efektif untuk skenario pertempuran di mana keputusan lokal per turn sudah cukup untuk mengakumulasi skor tertinggi dalam durasi pertempuran yang terbatas. Namun pendekatan ini memiliki keterbatasan dalam situasi kompleks seperti pertempuran multi-bot di mana evaluasi target secara dinamis dan perencanaan jangka panjang diperlukan. Meskipun demikian, Nemesis menunjukkan efisiensi tinggi dalam pengambilan keputusan dan kemampuan beradaptasi yang baik melalui mekanisme timeout dan reset target secara otomatis.

5.2 Saran

Untuk meningkatkan performa dan fleksibilitas ENAA_Nemesis, terdapat beberapa saran pengembangan yang dapat diimplementasikan pada iterasi berikutnya:

a. Implementasi Multi-Target Evaluation

Saat ini bot mengunci musuh pertama yang terdeteksi tanpa mempertimbangkan apakah target tersebut adalah pilihan terbaik. Sistem pemilihan target dapat ditingkatkan dengan mengevaluasi seluruh musuh yang terdeteksi sebelum mengunci satu target, misalnya dengan mempertimbangkan:

- Energi musuh saat ini, diprioritaskan musuh dengan energi terendah untuk memaksimalkan peluang Bullet Damage Bonus
- Jarak ke musuh, diprioritaskan musuh terdekat untuk memaksimalkan efektivitas firepower tinggi
- Ancaman musuh, diprioritaskan musuh yang paling aktif menembak ke arah Nemesis

Dengan pendekatan ini, keputusan target locking menjadi lebih kontekstual dan tidak hanya bergantung pada siapa yang pertama masuk radar.

b. Penambahan Energy Threshold untuk Survival

Bot saat ini tidak memiliki mekanisme khusus untuk kondisi energi kritis. Sistem manajemen energi dapat ditingkatkan dengan menambahkan threshold misalnya saat energi di bawah 20, di mana bot berhenti menembak dan sepenuhnya fokus pada survival seperti mempercepat gerakan melingkar, menjauh dari musuh, dan menunggu energi cukup aman sebelum kembali menyerang. Pendekatan ini dapat memaksimalkan Survival Score yang bernilai 50 poin setiap kali musuh lain mati, daripada terus menembak dengan risiko habis energi lebih cepat.

c. Optimasi Toleransi Penembakan Adaptif

Threshold penembakan saat ini ditetapkan tetap di 15° untuk semua jarak. Sistem ini dapat dioptimalkan dengan membuat threshold bersifat adaptif berdasarkan jarak:

- Jarak sangat dekat ($< 150\text{px}$) maka perlonggar threshold hingga 25° karena peluru cepat sampai dan musuh tidak sempat bergerak jauh

- Jarak menengah (150–400px) maka pertahankan 15° sebagai keseimbangan
- Jarak jauh (> 400px) maka perketat threshold hingga 5° karena peluru butuh waktu lama dan akurasi sangat krusial

Pendekatan ini meningkatkan response time di jarak dekat sekaligus menjaga akurasi di jarak jauh.

d. Eksplorasi Prediksi Non-Linear

Prediksi posisi musuh saat ini mengasumsikan musuh terus bergerak dengan kecepatan dan arah yang konstan. Akurasi prediksi dapat ditingkatkan dengan memperhitungkan:

- Akselerasi musuh berdasarkan perubahan kecepatan antar scan
- Pola manuver musuh yang teramati selama pertempuran, seperti apakah musuh cenderung berbelok atau berbalik arah secara berulang.
- Probabilitas musuh menabrak dinding berdasarkan posisi dan arah geraknya saat ini

Prediksi non-linear ini akan meningkatkan hit rate secara signifikan terutama menghadapi bot dengan gerakan yang tidak beraturan seperti ENAA_Predator.

e. Implementasi Wall Avoidance Proaktif

Saat ini Nemesis hanya merespons tabrakan dinding secara reaktif melalui OnHitWall() dengan SetBack(40), yang memutus momentum circular movement. Sistem ini dapat ditingkatkan dengan deteksi dinding proaktif setiap turn bot menghitung jarak ke dinding terdekat berdasarkan posisi dan arah gerak saat ini, lalu menyesuaikan sudut belok sebelum tabrakan terjadi. Dengan pendekatan ini, circular movement dapat dipertahankan tanpa interupsi dan bot tidak kehilangan momentum ofensif hanya karena menabrak batas arena.

f. Pengujian dan Simulasi Lanjutan

Untuk memvalidasi efektivitas pengembangan di atas, disarankan pengujian dilakukan pada berbagai kondisi tambahan:

- Pertempuran dengan jumlah bot lebih banyak (5–10 bot) untuk menguji ketahanan target locking di arena yang lebih ramai

- Skenario dengan bot lawan yang memiliki strategi evasif ekstrem untuk menguji batas akurasi predictive firing
- Pengujian dengan Turn Limit yang lebih ketat untuk mengevaluasi efisiensi energi dan survival strategy dalam waktu terbatas

LAMPIRAN

A. Repository Github

https://github.com/adelia124140123/TUBES-STIMA_ENAA.git

B. Video Penjelasan

<https://drive.google.com/file/d/13rfnhnC-84WriPDbLmrUn5TD6hflzBaj/view?usp=sharing>

DARFAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

Heineman, G. T. (n.d.). *Robocode Tank Royale documentation*. <https://robocode-dev.github.io/tank-royale/>